

# 决策树

## 概念

分类决策树是一种描述对实例进行分类的树形结构，是通过一系列规则对数据进行分类的过程。

## 构建步骤

1. 构建根节点
2. 选择最优特征，以此分类训练数据集
3. 若子集被基本正确分类，构建叶节点，否则继续选择新的最优特征。
4. 重复上述两步，直到所有训练数据子集被正确分类

## 基本学习方法

- 本质：从训练数据集中归纳出一组分类规则，与训练数据集不相矛盾
- 假设空间：由无穷多个条件概率模型组成
- 一棵好的决策树：与训练数据矛盾较小，同时具有很好的泛化能力
- 策略：最小化损失函数
- 特征选择：递归选择最优特征
- 生成：对应特征空间的划分，直到所有训练子集被基本正确分类
- 剪枝：避免过拟合，具有更好的泛化能力

## 特征选择： 信息增益

### 信息增益

输入：训练数据集  $D$  和特征  $A$

输出：特征  $A$  对  $D$  的信息增益  $g(D, A)$

- 计算经验熵  $H(D)$ :

$$H(D) = - \sum_{k=1}^K \frac{|C_k|}{|D|} \log_2 \frac{|C_k|}{|D|}$$

- 计算经验条件熵  $H(D|A)$ :

$$H(D|A) = \sum_{i=1}^n \frac{|D_i|}{|D|} H(D_i) = - \sum_{i=1}^n \frac{|D_i|}{|D|} \sum_{k=1}^K \frac{|D_{ik}|}{|D_i|} \log_2 \frac{|D_{ik}|}{|D_i|}$$

- 计算信息增益:

$$g(D, A) = H(D) - H(D|A)$$

注意此处条件熵中**第二个等号并不成立**， $H(D_i)$  化简出来的结果中分母应该为  $D_i$ 。

## 信息增益比

$$\text{Gain\_ratio}(D, a) = \frac{\text{Gain}(D, a)}{\text{IV}(a)},$$

其中

$$\text{IV}(a) = - \sum_{v=1}^V \frac{|D^v|}{|D|} \log_2 \frac{|D^v|}{|D|}$$

当 $\alpha$ 属性的取值越多时  
 $\text{IV}(\alpha)$ 值越大

为了防止因为特征对应的取值空间大小影响选择结果，因此引入信息增益比。

选择信息增益还是选择信息增益比要根据实际情况来进行判断。例如：承受风险能力越强就选择信息增益，否则选择信息增益比

## 决策树生成

### ID3算法

算法总览：

输入：训练数据集  $D$ 、特征集  $A$ 、阈值  $\epsilon$

输出：决策树  $T$

- 判断  $T$  是否需要选择特征生成决策树；
  - 若  $D$  中所有实例属于同一类，则  $T$  为单结点树，记录实例类别  $C_k$ ，以此作为该结点的类标记，并返回  $T$ ；
  - 若  $D$  中所有实例无任何特征 ( $A = \emptyset$ )，则  $T$  为单结点树，记录  $D$  中实例个数最多类别  $C_k$ ，以此作为该结点的类标记，并返回  $T$ ；
- 否则，计算  $A$  中各特征的信息增益，并选择信息增益最大的特征  $A_g$ ；
  - 若  $A_g$  的信息增益小于  $\epsilon$ ，则  $T$  为单结点树，记录  $D$  中实例个数最多类别  $C_k$ ，以此作为该结点的类标记，并返回  $T$ ；
  - 否则，按照  $A_g$  的每个可能值  $a_i$ ，将  $D$  分为若干非空子集  $D_i$ ，将  $D_i$  中实例个数最多的类别作为标记，构建子结点，以结点和其子节点构成  $T$ ，并返回  $T$ ；
- 第  $i$  个子结点，以  $D_i$  为训练集， $A - A_g$  为特征集合，递归地调用以上步骤，得到子树  $T_i$  并返回。

针对于判断  $T$  是否需要选择特征生成树，两种不需要向下进行的分别是：

- 无需分类，因为所有实例按照定义都属于同一类别。比如：希望能根据特征（如湿度，温度等）判断晴天雨天，如果这一实例中全是晴天就无需再分类。
- 没有可以选择的能够有区分度的特征了

阈值的  $\epsilon$  代表，如果信息增益小于这个阈值，就不进行分类。

### C4.5算法

算法总览：

输入：训练数据集  $D$ 、特征集  $A$ 、阈值  $\epsilon$   
输出：决策树  $T$

- 判断  $T$  是否需要选择特征生成决策树
  - 若  $D$  中所有实例属于同一类，则  $T$  为单结点树，记录实例类别  $C_k$ ，以此作为该结点的类标记，并返回  $T$ ；
  - 若  $D$  中所有实例无任何特征 ( $A = \emptyset$ )，则  $T$  为单结点树，记录  $D$  中实例个数最多类别  $C_k$ ，以此作为该结点的类标记，并返回  $T$ ；
- 否则，计算  $A$  中各特征的信息增益比，并选择信息增益比最大的特征  $A_g$ ：
  - 若  $A_g$  的信息增益比  $\geq \epsilon$ ，则  $T$  为单结点树，记录  $D$  中实例个数最多类别  $C_k$ ，以此作为该结点的类标记，并返回  $T$ ；
  - 否则，按照  $A_g$  的每个可能值  $a_j$ ，将  $D$  分为若干非空子集  $D_j$ ，将  $D_j$  中实例个数最多的类别作为标记，构建子结点，以结点和其子节点构成  $T$ ，并返回  $T$ ；
- 第  $i$  个子结点，以  $D_j$  为训练集， $A - A_g$  为特征集合，递归地调用以上步骤，得到子树  $T_j$  并返回。

最主要的特征是采用**信息增益比**而不是信息增益，同时可以**处理连续的数值**。

## 剪枝

优秀的决策树：

在具有好的拟合和泛化能力的同时，

- 深度小（所有节点的最大层次数）
- 叶节点少
- 深度小并且叶结点少

## 预剪枝

生成过程中，对每一个结点划分前进行估计。若当前节点的划分不能提升泛化能力，则停止划分，记当前节点为叶节点。

- 限定决策树的深度
- 设定一个阈值（C4.5算法）
- 设置某个指标，比较结点划分前后的泛化能力

利用测试集对训练出来的决策树从上到下逐节点进行评估，计算其评估出来的误差率，然后比较不分裂和分裂的误差率。（注意针对单个节点分裂时，其分裂出来的节点应该全为叶节点，根据实例占比来决定该叶节点应该属于哪个结果）

## 后剪枝

生成一颗完整的决策树之后，自底向上对内部节点考察。若此内部节点变为叶节点，可以提升泛化能力，则做此替换。

- 降低错误剪枝(REP)
  - 自下而上，使用测试集来剪枝。对每个节点，计算剪枝前和剪枝后的误判个数，若是剪枝有利于减少误判（包括相等的情况），则减掉该结点所在分支。
    - 计算复杂度低
    - 操作简单
    - 受测试集影响大，如果测试集比训练集小很多，会限制分类的精度。

## • 悲观错误剪枝(PEP)

只需要训练集不需要验证集，而且是自上而下地进行剪枝的。

步骤：

- 计算剪枝前目标子树每个叶子结点的误差，并进行连续修正：

$$Error(Leaf_i) = \frac{error(Leaf_i) + 0.5}{N(T)}$$

- 计算剪枝前目标子树的修正误差：

$$Error(T) = \sum_{i=1}^L Error(Leaf_i) = \frac{\sum_{i=1}^L error(Leaf_i) + 0.5 * L}{N(T)}$$

- 计算剪枝前目标子树误判个数的期望值：

$$E(T) = N(T) \times Error(T) = \sum_{i=1}^L error(Leaf_i) + 0.5 * L$$

- 计算剪枝前目标子树误判个数的标准差：

$$std(T) = \sqrt{N(T) \times Error(T) \times (1 - Error(T))}$$

- 计算剪枝前误判上限（即悲观误差）：

$$E(T) + std(T)$$

- 计算剪枝后该结点的修正误差：

$$Error(Leaf) = \frac{error(Leaf) + 0.5}{N(T)}$$

- 计算剪枝后该结点误判个数的期望值：

$$E(Leaf) = error(Leaf) + 0.5$$

- 比较剪枝前后的误判个数，如果满足以下不等式，则剪枝；否则，不剪枝。

$$E(Leaf) < E(T) + std(T)$$

## • 最小误差剪枝(MEP)

自下而上剪枝，只需要训练集。

根据剪枝前后的最小分类错误概率来决定是否剪枝。

- 在结点  $T$  处，属于类别  $k$  的概率

$$P_k(T) = \frac{n_k(T) + Pr_k(T) \times m}{N(T) + m}$$

- 在结点  $T$  处，预测错误率

$$Error(T) = \min\{1 - P_k(T)\} = \min\left\{\frac{N(T) - n_k(T) + m(1 - Pr_k(T))}{N(T) + m}\right\}$$

实际算法：

$$Error(T) = \frac{N(T) - n_k(T) + K - 1}{N(T) + K}$$

步骤：

- 计算剪枝前目标子树每个叶子结点的预测错误率：

$$Error(Leaf_i) = \frac{N(Leaf_i) - n_k(Leaf_i) + K - 1}{N(Leaf_i) + K}$$

- 计算剪枝前目标子树的预测错误率：

$$Error(Leaf) = \sum_{i=1}^L w_i Error(Leaf_i) = \sum_{i=1}^L \frac{N(Leaf_i)}{N(T)} Error(Leaf_i)$$

- 计算剪枝后目标子树的预测错误率：

$$Error(T) = \frac{N(T) - n_k(T) + K - 1}{N(T) + K}$$

- 比较剪枝前后的预测错误率，如果满足以下不等式，则剪枝；否则，不剪枝。

$$Error(T) < Error(Leaf)$$

## • 基于错误的剪枝(EBP)

根据剪枝前后的误判个数来决定是否剪枝。自下而上剪枝，只需要训练集即可。

置信水平  $\alpha$  下每个节点的误判率上界：

$$U_\alpha(e(T), N(T)) = \frac{e(T) + 0.5 + \frac{q_\alpha^2}{2} + q_\alpha \sqrt{\frac{(e(T) + 0.5)(N(T) - e(T) - 0.5)}{N(T)} + \frac{q_\alpha^2}{4}}}{N(T) + q_\alpha^2}$$

训练的具体步骤：

步骤:

- 计算剪枝前目标子树每个叶子结点的误判率上界:

$$U_{CF}(e(Leaf_i), N(Leaf_i)) = \frac{e(Leaf_i) + 0.5 + \frac{q_\alpha^2}{2} + q_\alpha \sqrt{\frac{(e(Leaf_i) + 0.5)(N(Leaf_i) - e(Leaf_i) - 0.5)}{N(Leaf_i)} + \frac{q_\alpha^2}{4}}}{N(Leaf_i) + q_\alpha^2}$$

- 计算剪枝前目标子树的误判个数上界:

$$E(Leaf) = \sum_{i=1}^L N(Leaf_i) U_{CF}(e(Leaf_i), N(Leaf_i))$$

- 计算剪枝后目标子树的误判率上界:

$$U_{CF}(e(T), N(T)) = \frac{e(T) + 0.5 + \frac{q_\alpha^2}{2} + q_\alpha \sqrt{\frac{(e(T) + 0.5)(N(T) - e(T) - 0.5)}{N(T)} + \frac{q_\alpha^2}{4}}}{N(T) + q_\alpha^2}$$

- 计算剪枝后目标子树的误判个数上界:

$$E(T) = N(T) U_{CF}(e(T), N(T))$$

- 比较剪枝前后的误判个数上界, 如果满足以下不等式, 则剪枝; 否则, 不剪枝。

$$E(T) < E(Leaf)$$

## • 代价-复杂度剪枝(CCP)

损失函数:

$$C_\alpha = C(T) + \alpha |T|$$

其中,  $C(T) = \sum_{t=1}^{|T|} N_t H_t(T) = - \sum_{t=1}^{|T|} \sum_{k=1}^K N_{tk} \log \frac{N_{tk}}{N_t}$

正则化一般形式:

$$\min_{f \in \mathcal{F}} \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i)) + \lambda J(f)$$

其中T为决策树,  $C(T)$ 为预测误差,  $|T|$ 是树叶节点个数,  $\alpha$ 代表复杂度, 用来平衡拟合能力和泛化能力。

具体步骤:

步骤:

- 剪枝前的决策树记做  $T_A$ , 计算每个叶子结点  $t$  的经验熵:

$$H_t = - \sum_{k=1}^K \frac{N_{tk}}{N_t} \log \frac{N_{tk}}{N_t}$$

- 剪枝前决策树的损失函数为:

$$C_\alpha(T_A) = C(T_A) + \alpha |T_A|$$

- 剪枝后的决策树记做  $T_B$ , 损失函数为:

$$C_\alpha(T_B) = C(T_B) + \alpha |T_B|$$

- 比较剪枝前后的损失函数, 如果满足以下不等式, 则剪枝; 否则, 不剪枝。

$$C_\alpha(T_B) \leq C_\alpha(T_A)$$

# CART算法

## 基尼指数

基尼指数

假设现在有  $K$  个类，样本点属于第  $k$  个类的概率为  $p_k$ ，则概率分布的基尼指数为：

$$Gini(p) = \sum_{k=1}^K p_k(1 - p_k) = 1 - \sum_{k=1}^K p_k^2$$

二分类：

$$Gini(p) = 2p(1 - p)$$

样本集  $D$  的基尼指数：

$$Gini(D) = 1 - \sum_{k=1}^K \left(\frac{|C_k|}{|D|}\right)^2$$

如何基于基尼指数来进行判别：

特征A 条件下，样本集  $D$  的基尼指数为：

$$Gini(D, A) = \frac{|D_1|}{|D|} Gini(\underline{D_1}) + \frac{|D_2|}{|D|} Gini(\underline{D_2})$$

Y : {好吃, 不好吃}

10      5      5

$Gini(D) = 2 \times \frac{1}{2} \times \frac{1}{2} = \frac{1}{2} = 0.5$

基尼指数越小，分类后的结果更加确定，分类效果越好

## 具体算法

### 分类树

输入：训练数据集  $D$ 、特征集  $A$ 、阈值  $\epsilon$

输出：CART决策树  $T$

- 从根节点出发，进行操作，构建二叉树；
- 结点处的训练数据集为  $D$ ，计算现有特征对该数据集的基尼指数，并选择最优特征。
  - 在特征  $A_g$  下，对其可能取的每个值  $a_g$ ，根据样本点对  $A_g = a_g$  的测试为“是”或“否”，将  $D$  分割成  $D_1$  和  $D_2$  两部分，计算  $A_g = a_g$  时的基尼指数。
  - 选择基尼指数最小的那个值作为该特征下的最优切分点。
  - 计算每个特征下的最优切分点，并比较在最优切分下的每个特征的基尼指数，选择基尼指数最小的那个特征，即最优特征。
- 根据最优特征与最优切分点，从现结点生成两个子结点，将训练数据集依特征分配到两个子结点中去。
- 分别对两个子结点递归地调用上述步骤，直至满足停止条件（比如结点中的样本个数小于预定阈值，或样本集的基尼指数小于预定阈值（样本基本属于同一类），或者没有更多特征），即生成CART决策树。

### 回归树

由于回归任务是连续的，所以必须切分成多个块才可以使用决策树。

假设将输入空间划分为 $M$ 个单元 $R_1, R_2, \dots, R_M$ ，并在每个单元 $R_m$ 上有一个固定的输出值 $c_m$ 。回归树模型可表示为

$$f(x) = \sum_{m=1}^M c_m I(x \in R_m)$$

平方误差：

$$\sum_{x_i \in R_m} (y_i - f(x_i))^2$$

最优输出：

$$\hat{c}_m = \text{average}(y_i | x_i \in R_m)$$

选择第 $x^{(j)}$ 个变量和取值 $s$ ，分别作为切分变量和切分点，并定义两个区域：

$$R_1(j, s) = \{x | x^{(j)} \leq s\} \quad R_2(j, s) = \{x | x^{(j)} > s\}$$

寻找最优切分变量 $j$ 和最优切分点 $s$ ：

$$\min_{j, s} \left[ \min_{c_1} \sum_{x_i \in R_1(j, s)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j, s)} (y_i - c_2)^2 \right]$$

对固定输入变量 $j$ 可以找到最优切分点 $s$ ：

$$\hat{c}_1 = \text{ave}(y_i | x_i \in R_1(j, s)) \quad \text{和} \quad \hat{c}_2 = \text{ave}(y_i | x_i \in R_2(j, s))$$

## 具体算法流程

输入：训练数据集  $D$ ，停止条件

输出：CART决策树  $T$

- 从根节点出发，进行操作，构建二叉树；
- 结点处的训练数据集为  $D$ ，计算变量的最优切分点，并选择最优变量。

$$\min_{j, s} \left[ \min_{c_1} \sum_{x_i \in R_1(j, s)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j, s)} (y_i - c_2)^2 \right]$$

- 在  $j$  变量下，对其可能取的每个值  $s$ ，根据样本点分割成  $R_1$  和  $R_2$  两部分，计算切分点为  $s$  时的平方误差。
- 选择平方误差最小的那个值作为该变量下的最优切分点。
- 计算每个变量下的最优切分点，并比较在最优切分下的每个变量的平方误差，选择平方误差最小的那个变量，即最优变量。

- 根据最优特征与最优切分点  $(j, s)$ ，从现结点生成两个子结点，将训练数据集依变量配到两个子结点中去，得到相应的输出值。

$$R_1(j, s) = \{x | x^{(j)} \leq s\}, \quad R_2(j, s) = \{x | x^{(j)} > s\}$$
$$\hat{c}_m = \frac{1}{N_m} \sum_{x_i \in R_m(j, s)} y_i, \quad x \in R_m, \quad m = 1, 2$$

- 继续对两个子区域调用上述步骤，直至满足停止条件，即生成CART 决策树。

$$f(x) = \sum_{m=1}^M \hat{c}_m I(x \in R_m)$$

## 剪枝

利用ccp（代价-复杂度剪枝）来进行剪枝

算法流程：

输入：CART算法生成的决策树。

输出：最优决策树  $T_\alpha$ 。

- 设  $k = 0$ ,  $T = T_0$ 。
- 设  $\alpha = +\infty$ 。
- 自下而上地对各内部结点  $t$  计算  $C(T_t)$ ,  $|T_t|$  以及

$$g(t) = \frac{C(t) - C(T_t)}{|T_t| - 1}, \quad \alpha = \min(\alpha, g(t))$$

这里， $T_t$ 表示以 $t$ 为根结点的子树， $C(T_t)$ 是对训练数据的预测误差， $|T_t|$ 是 $T_t$ 的叶结点个数。

(4)自下而上地访问内部结点 $t$ ，如果有 $g(t) = \alpha$ ，进行剪枝，并对叶结点 $t$ 以多数类法决定其类，得到树 $T$ 。

(5)设 $k = k + 1$ ， $\alpha_k = \alpha$ ， $T_k = T$ 。

(6)如果 $T$ 不是由根结点单独构成的树，则回到步骤(4)。

(7)采用交叉验证法在于树序列 $T_0, T_1, \dots, T_n$ 中选取最优子树 $T_\alpha$ 。